Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Seminario de Tecnologías de Información 2026

Entrega 10
Implementación de DevOps 2

Integración Continua, Entrega Continua y Plan de Pruebas

Repositorio GitHub: https://github.com/lorh2025/fiver-cable-devops

Empresa objeto de estudio: Fiver Cable

Integrantes del grupo

Nombre completo	Número de carné
Hazel Ruvi Perez Carcamo	0900-21-13727
Byron Enrique Morales	0900-1015162
Luis Oswaldo Reyes	0900-98-2342

Guatemala, martes 29 de septiembre de 2026

Índice

1. Introducción.....3
2. Tecnologías y arquitectura implementada.....3
3. Integración y entrega continua.....5
3.1 Flujo de trabajo con ramas.....5
3.2 Etapas del pipeline.....5
3.3 Definición del pipeline (azure-pipelines.yml).....5
3.4 Despliegue a staging en Azure.....6
3.5 Evidencia de ejecuciones exitosas y fallidas.....7
4. Plan de pruebas.....9
4.1 Objetivos.....9
4.2 Propósito.....9
4.3 Alcance.....9
4.4 Tipos de pruebas.....9
4.5 Estrategia de pruebas.....10
4.6 Casos de prueba.....11
4.7 Ejecución del plan de pruebas.....13
4.8 Parámetros del sistema.....14
5. Evidencia Scrum del Sprint 2.....15
5.1 Sprint Backlog (tareas de 16 horas o menos).....15
5.2 Burn down chart.....15
5.3 Daily Scrum.....16
5.4 Revisión del Sprint.....16
5.5 Retrospectiva.....16
6. Video YouTube #2: guion.....17
7. Conclusiones.....17
Anexo A. Configuración en Azure DevOps.....17

1. Introducción

En el Entregable 9 (DevOps 1) se dejó configurada la base del ciclo DevOps de Fiver Cable: el repositorio en GitHub sincronizado con Azure Repos, los Dockerfiles por microservicio, el tablero Scrum en Azure Boards, un pipeline inicial de build y los dos primeros casos en Azure Test Plans. Los contenedores de esa entrega respondían únicamente un endpoint /health de demostración.

En esta segunda entrega los cuatro microservicios pasan a tener código real conectado a la base de datos del Entregable 8, y el pipeline evoluciona a un flujo completo de Integración Continua y Entrega Continua con las etapas checkout, build, test y despliegue a staging. Sobre ese flujo se ejecuta un Plan de Pruebas de 28 casos (26 automatizados con pytest y Selenium y 2 manuales de experiencia de usuario) cuyo reporte HTML se genera en cada ejecución.

El proceso que se automatiza es el mismo que el grupo seleccionó como punto de partida de la transformación digital: el cobro digital en campo. El técnico cobra la factura FEL desde su teléfono, la base de datos marca la factura como pagada mediante el trigger trg_procesar_pago_exitoso y el servicio del cliente queda listo para reactivarse sin pasar por la oficina.

2. Tecnologías y arquitectura implementada

La guía del seminario pide un lenguaje de programación y un sistema gestor de base de datos libres. Todas las herramientas elegidas son de código abierto o gratuitas para el alcance académico del proyecto:

Componente	Tecnología	Licencia	Uso en el proyecto
Lenguaje	Python 3.11	PSF (libre)	Código de los 4 microservicios y de las pruebas
Framework web	FastAPI 0.142	MIT	API REST y documentación OpenAPI en /docs
SGBD	PostgreSQL 16	PostgreSQL (libre)	fivercable_db con 4 esquemas (database-per-service)
Contenedores	Docker y Docker Compose	Apache 2.0	Entorno idéntico en local, CI y staging
Pruebas	pytest, pytest-html, pytest-cov	MIT	Ejecución, reporte HTML y cobertura
Pruebas de interfaz	Selenium 4 + Chrome	Apache 2.0	Pruebas de aceptación del portal
CI/CD	Azure Pipelines (y Jenkinsfile equivalente)	Gratuito para proyectos	Checkout → Build → Test → Deploy Staging
Nube	Azure Container Apps + Azure Database for PostgreSQL	Azure for Students	Entorno de staging

En el Informe de Consistencia se había señalado que DevOps 1 usaba nombres de servicio (auth, catálogo, pedidos, pagos) distintos a los esquemas del modelo relacional. Para no romper lo ya

entregado se conservan los nombres de DevOps 1 y se fija de forma explícita, en el código y en la documentación, qué esquema pertenece a cada servicio:

Microservicio	Puerto	Esquema	Funcionalidad implementada en el Sprint 2
ms_auth	3000	usuarios_db	Registro con validación de NIT y contraseña, login con JWT, roles, cambio de estado auditado por trigger
ms_catalogo	3001	red_db	Planes de servicio, registro de ONUs con MAC única, clasificación de señal óptica, procedimiento de estado OLT
ms_pedidos	3002	soporte_db	Tickets con prioridad automática y SLA, cierre con sp_cerrar_ticket, vista de pendientes
ms_pagos	3003	billing_db	Facturas FEL, registro de pagos con validación de monto, cierre diario y portal de cobro en campo

Cada servicio separa sus reglas de negocio puras (archivo logic.py) de la capa de API (main.py). Esta separación es la que permite probar las reglas con pruebas unitarias rápidas, sin base de datos, en la etapa de Build del pipeline.

3. Integración y entrega continua

3.1 Flujo de trabajo con ramas

El equipo trabaja con tres tipos de rama. Ningún cambio llega a main sin pasar antes por el pipeline completo:

- **feature/<tarea>:** una rama por tarea del Sprint Backlog (por ejemplo feature/portal-cobro).
- **develop:** integración del sprint. Cada Pull Request hacia develop ejecuta Build y Test.
- **main:** versión estable. Solo los cambios en main se despliegan automáticamente a staging.

3.2 Etapas del pipeline

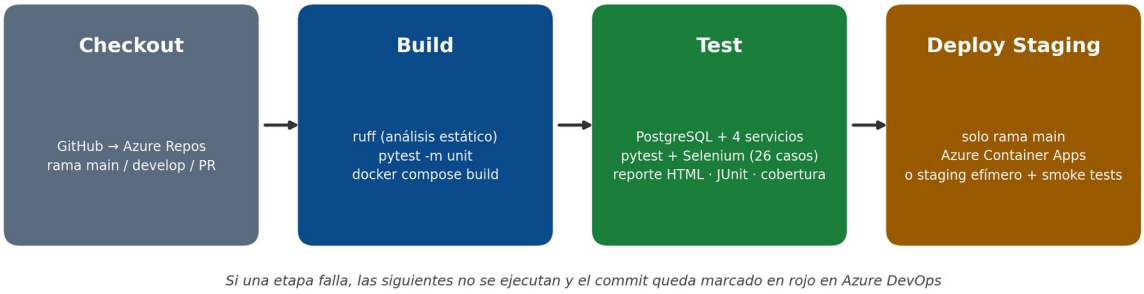


Figura 1. Etapas del pipeline CI/CD de Fiver Cable en Azure Pipelines.

Etapa	Qué hace	Criterio para continuar
Checkout	Descarga el commit desde GitHub/Azure Repos en un agente ubuntu-latest limpio	Código disponible
Build	Instala dependencias, análisis estático con ruff, 20 pruebas unitarias y construcción de las 4 imágenes Docker	Sin errores de estilo, unitarias al 100 % e imágenes construidas
Test	Levanta PostgreSQL con el script del Entregable 8 y los 4 servicios; ejecuta la suite completa; publica JUnit, cobertura y el reporte HTML como artefacto	43 ejecuciones aprobadas
Deploy Staging	Solo en main. Despliega en el Environment fivercable-staging (Azure Container Apps o entorno efímero) y ejecuta smoke tests	Los 4 /health en OK y pruebas de sistema aprobadas

3.3 Definición del pipeline (azure-pipelines.yml)

El pipeline se define como código en la raíz del repositorio, de modo que cualquier cambio al proceso queda versionado y revisado como el resto del código. Extracto de la etapa de pruebas:

```
- stage: Test
  dependsOn: Build
  jobs:
  - job: test
    steps:
```

```
- script: docker compose up -d --build --wait
  displayName: Levantar entorno de pruebas (PostgreSQL + 4 microservicios)
- script: |
    python -m pytest --junitxml=$(REPORTS_DIR)/junit.xml \
      --html=$(REPORTS_DIR)/reporte-pruebas.html --self-contained-html \
      --cov=src --cov-report=xml:$(REPORTS_DIR)/coverage.xml
  displayName: Ejecutar Plan de Pruebas completo (pytest + Selenium)
- task: PublishTestResults@2        # pestaña Tests de Azure DevOps
- task: PublishCodeCoverageResults@2
- task: PublishPipelineArtifact@1   # reporte HTML y capturas de Selenium
```

La guía menciona Jenkins y GitLab CI como ejemplos. Como el equipo ya trabaja en Azure DevOps desde DevOps 1, el pipeline principal es de Azure Pipelines; aun así, el repositorio incluye un Jenkinsfile con las mismas cuatro etapas, lo que demuestra que el proceso no depende de una herramienta específica.

3.4 Despliegue a staging en Azure

La etapa Deploy Staging usa un Environment de Azure DevOps (fivercable-staging), que guarda el historial de despliegues y permite agregar aprobaciones manuales antes de publicar. Tiene dos modalidades:

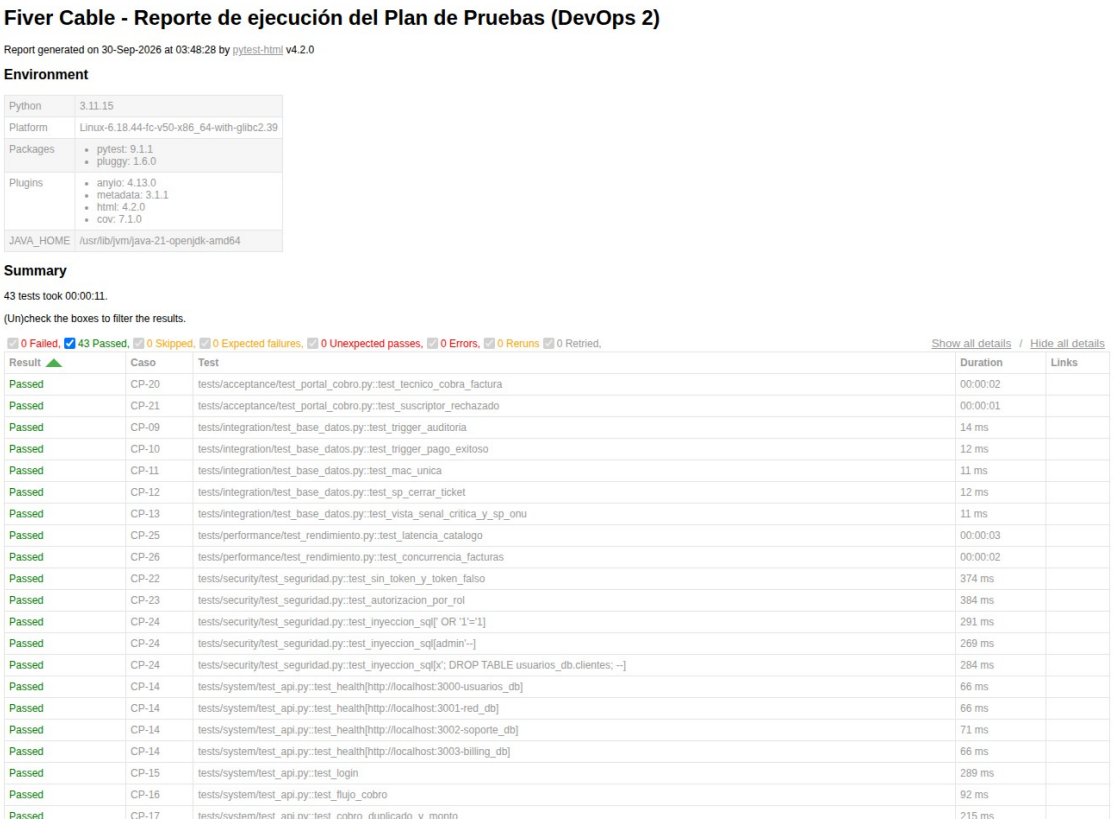
- **Staging efímero (predeterminado):** en cada cambio a main se levanta el sistema completo en el agente con Docker Compose y se ejecutan las pruebas de sistema como smoke tests.
- **Staging en Azure:** al ejecutar el pipeline con el parámetro «Desplegar Staging en Azure Container Apps», el script infra/azure/deploy-staging.sh crea o actualiza los recursos siguientes y ejecuta los smoke tests contra las URLs públicas.

Recurso de Azure	Nombre	Propósito
Grupo de recursos	rg-fivercable-staging	Agrupa todo el entorno para administrarlo y eliminarlo junto
Azure Container Registry (Basic)	acrfivercableumg2026	Almacena las imágenes de los 4 microservicios versionadas con el número de build
Azure Database for PostgreSQL Flexible (B1ms)	pg-fivercable-umg2026	fivercable_db con el script del Entregable 8, conexión TLS obligatoria
Azure Container Apps (entorno + 4 apps)	ms-auth, ms-catalogo, ms-pedidos, ms-pagos	Ejecución de los contenedores con HTTPS y escala de 0 a 2 réplicas

Las credenciales nunca se escriben en el repositorio: la contraseña de PostgreSQL y el secreto JWT de staging se guardan como variables secretas del pipeline (PG_ADMIN_PASSWORD, JWT_SECRET_STAGING) y llegan a los contenedores como secretos de Container Apps.

3.5 Evidencia de ejecuciones exitosas y fallidas

La Figura 2 muestra la ejecución completa del plan: 43 ejecuciones (26 casos, algunos parametrizados) aprobadas en 11 segundos, con la columna Caso que enlaza cada prueba con su ID del plan.



Fiver Cable - Reporte de ejecución del Plan de Pruebas (DevOps 2)

Report generated on 30-Sep-2026 at 03:48:28 by pytest-html v4.2.0

Environment

Python	3.11.15
Platform	Linux-6.18.44-fc-v50-x86_64-with-glibc2.39
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• anyio: 4.13.0 • metadata: 3.1.1 • html: 4.2.0 • cov: 7.1.0
JAVA_HOME	/usr/lib/jvm/java-21-openjdk-amd64

Summary

43 tests took 00:00:11.

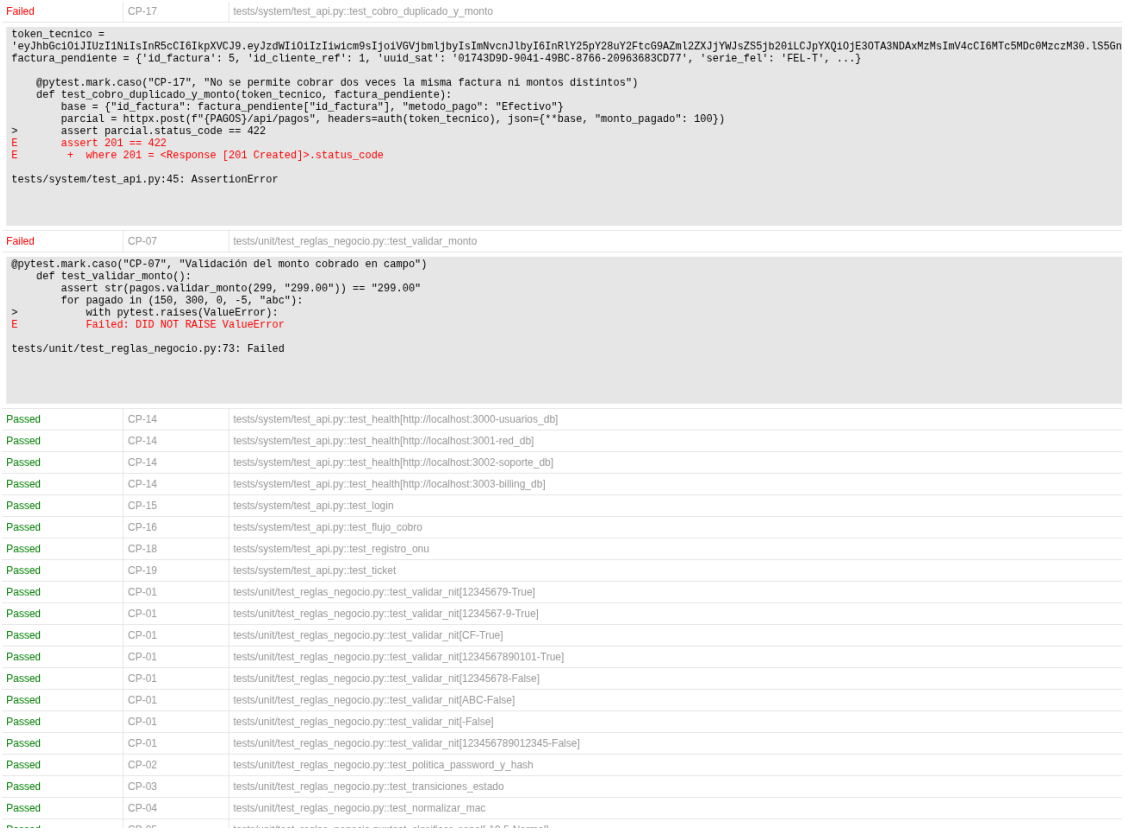
(Un)check the boxes to filter the results.

0 Failed, 43 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Caso	Test	Duration	Links
Passed	CP-20	tests/acceptance/test_portal_cobro.py::test_tecnico_cobra_factura	00:00:02	
Passed	CP-21	tests/acceptance/test_portal_cobro.py::test_suscriptor_rechazado	00:00:01	
Passed	CP-09	tests/integration/test_base_datos.py::test_trigger_auditoria	14 ms	
Passed	CP-10	tests/integration/test_base_datos.py::test_trigger_pago_exitoso	12 ms	
Passed	CP-11	tests/integration/test_base_datos.py::test_mac_unica	11 ms	
Passed	CP-12	tests/integration/test_base_datos.py::test_sp_cerrar_ticket	12 ms	
Passed	CP-13	tests/integration/test_base_datos.py::test_vista_senal_critica_y_sp_onu	11 ms	
Passed	CP-25	tests/performance/test_rendimiento.py::test_latencia_catalogo	00:00:03	
Passed	CP-26	tests/performance/test_rendimiento.py::test_concurrencia_facturas	00:00:02	
Passed	CP-22	tests/security/test_seguridad.py::test_sin_token_y_token_falso	374 ms	
Passed	CP-23	tests/security/test_seguridad.py::test_autorizacion_por_rol	384 ms	
Passed	CP-24	tests/security/test_seguridad.py::test_inyeccion_sql[' OR '1'='1]	291 ms	
Passed	CP-24	tests/security/test_seguridad.py::test_inyeccion_sql[admin'--]	269 ms	
Passed	CP-24	tests/security/test_seguridad.py::test_inyeccion_sql[x'; DROP TABLE usuarios_db.clientes; --]	284 ms	
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3000-usuarios_db]	66 ms	
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3001-red_db]	66 ms	
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3002-soporte_db]	71 ms	
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3003-billing_db]	66 ms	
Passed	CP-15	tests/system/test_api.py::test_login	289 ms	
Passed	CP-16	tests/system/test_api.py::test_flujo_cobro	92 ms	
Passed	CP-17	tests/system/test_api.py::test_cobro_duplicado_y_monto	215 ms	

Figura 2. Reporte HTML de pytest: ejecución exitosa del Plan de Pruebas (43 aprobadas, 0 fallidas).

Para comprobar que el pipeline detiene cambios defectuosos se simuló una regresión real: se modificó la regla de cobro para aceptar pagos parciales. La etapa de pruebas falló en los casos CP-07 (unitario) y CP-17 (sistema), por lo que el cambio no habría llegado a staging.



Failed	CP-17	tests/system/test_api.py::test_cobro_duplicado_y_monto

```
token_tecnico =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIzIiwicm9sIjoiVGVjbmljbyIsImNvcnJlbyI6InRlY25pY28uY2FtcG9AZml2ZXJjYWJsZS5jb20iLCJpYXQiOjE3OTA3NDAxMzMsImV4cCI6MTc5MDc0MzczM30.lS5Gn
factura_pendiente = {'id_factura': 5, 'id_cliente_ref': 1, 'uuid_sat': '01743D9D-9041-49BC-8766-20963683CD77', 'serie_fel': 'FEL-T', ...}

    @pytest.mark.caso("CP-17", "No se permite cobrar dos veces la misma factura ni montos distintos")
    def test_cobro_duplicado_y_monto(token_tecnico, factura_pendiente):
        base = {"id_factura": factura_pendiente["id_factura"], "metodo_pago": "Efectivo"}
        parcial = httpx.post(f"{PAGOS}/api/pagos", headers=auth(token_tecnico), json={**base, "monto_pagado": 100})
>       assert parcial.status_code == 422
E       assert 201 == 422
E        +  where 201 = <Response [201 Created]>.status_code

tests/system/test_api.py:45: AssertionError
```

Failed	CP-07	tests/unit/test_reglas_negocio.py::test_validar_monto

```
@pytest.mark.caso("CP-07", "Validación del monto cobrado en campo")
    def test_validar_monto():
        assert str(pagos.validar_monto(299, "299.00")) == "299.00"
        for pagado in (150, 300, 0, -5, "abc"):
>           with pytest.raises(ValueError):
E           Failed: DID NOT RAISE ValueError

tests/unit/test_reglas_negocio.py:73: Failed
```

Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3000-usuarios_db]
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3001-red_db]
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3002-soporte_db]
Passed	CP-14	tests/system/test_api.py::test_health[http://localhost:3003-billing_db]
Passed	CP-15	tests/system/test_api.py::test_login
Passed	CP-16	tests/system/test_api.py::test_flujo_cobro
Passed	CP-18	tests/system/test_api.py::test_registro_onu
Passed	CP-19	tests/system/test_api.py::test_ticket
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[12345679-True]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[1234567-9-True]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[CF-True]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[1234567890101-True]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[12345678-False]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[ABC-False]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[-False]
Passed	CP-01	tests/unit/test_reglas_negocio.py::test_validar_nit[123456789012345-False]
Passed	CP-02	tests/unit/test_reglas_negocio.py::test_politica_password_y_hash
Passed	CP-03	tests/unit/test_reglas_negocio.py::test_transiciones_estado
Passed	CP-04	tests/unit/test_reglas_negocio.py::test_normalizar_mac

Figura 3. Ejecución fallida: la regresión en la validación del monto es detectada por CP-17 y CP-07.

Además, al automatizar la creación de la base de datos el pipeline encontró un defecto real del script del Entregable 8: la instrucción SET time_zone es sintaxis de MySQL y PostgreSQL la rechaza. En pgAdmin pasó desapercibida porque la herramienta continúa ejecutando después del error; en el pipeline, con ON_ERROR_STOP, la creación se detuvo. Se corrigió a SET TIME ZONE 'America/Guatemala', lo que muestra el valor de ejecutar los scripts de forma automática y no solo manual.

4. Plan de pruebas

4.1 Objetivos

- Verificar que el proceso de cobro digital en campo funciona de extremo a extremo: autenticación del técnico, consulta de facturas pendientes, registro del pago y cambio de estado de la factura.
- Comprobar que los triggers, procedimientos y vistas del Entregable 8 se comportan como se documentó.
- Asegurar que solo los roles autorizados ejecutan operaciones sensibles y que el sistema resiste ataques comunes.
- Establecer una línea base de rendimiento y de cobertura de código para medir la mejora en los siguientes sprints.

4.2 Propósito

El plan convierte la calidad en algo medible y repetible: cada cambio al código ejecuta los mismos 28 casos y produce evidencia (reporte HTML, resultados JUnit y cobertura). Así el equipo y el catedrático pueden verificar en cualquier momento qué funciona, qué falló y en qué commit ocurrió.

4.3 Alcance

Dentro del alcance	Fuera del alcance (siguientes sprints)
Los 4 microservicios y sus endpoints REST	Integración real con la API de certificación FEL de la SAT
Script de base de datos: esquemas, triggers, procedimientos y vistas	Pasarela de pagos real (se usa código de autorización ingresado)
Portal web de cobro en campo	Bus de eventos RabbitMQ y reconexión automática en la OLT
Autenticación JWT y control de acceso por rol	Pruebas de carga sostenida y de estrés sobre Azure
Pipeline CI/CD y despliegue a staging	Aplicación móvil nativa

4.4 Tipos de pruebas

Tipo	Qué valida	Herramienta	Ubicación	Casos
Unitarias	Reglas de negocio puras (NIT, MAC, señal, montos, SLA)	pytest	tests/unit	CP-01 a CP-08
Integración	Triggers, procedimientos, vistas y restricciones en PostgreSQL	pytest + psycopg	tests/integration	CP-09 a CP-13
Sistema	API de los 4 servicios desplegados en contenedores	pytest + httpx	tests/system	CP-14 a CP-19
Aceptación	Flujo del usuario final en el portal	Selenium + Chrome	tests/acceptance	CP-20, CP-21
Seguridad	JWT, autorización por rol, inyección SQL, fuga de datos	pytest + PyJWT	tests/security	CP-22 a CP-24

Tipo	Qué valida	Herramienta	Ubicación	Casos
Rendimiento	Latencia p95 y concurrencia	pytest + hilos	tests/ performance	CP-25, CP-26
Usabilidad (manual)	Legibilidad y mensajes en pantalla de teléfono	Revisión guiada	Checklist	CP-27, CP-28

4.5 Estrategia de pruebas

- **Automatización primero:** todo lo que se puede verificar con una regla objetiva se automatiza con pytest; el flujo del portal se automatiza con Selenium en una ventana del tamaño de un teléfono.
- **Pirámide de pruebas:** muchas pruebas unitarias rápidas en Build (fallo en segundos), menos pruebas de integración y sistema en Test y pocas pruebas de interfaz, que son las más lentas.
- **Manual para UX:** la experiencia de uso (legibilidad, claridad de mensajes) se evalúa manualmente con un checklist, porque no se reduce a una regla automática.
- **Aislamiento de datos:** las pruebas de integración se ejecutan en una transacción que se revierte; las de sistema crean su propia factura, por lo que pueden repetirse sin limpiar la base.
- **Trazabilidad:** cada prueba lleva el ID de su caso (marcador @pytest.mark.caso), visible en el reporte HTML.
- **Criterios de entrada:** Build aprobado e imágenes construidas. Criterios de salida: 100 % de casos automatizados aprobados, cobertura total mayor o igual a 80 % y p95 dentro del umbral.

4.6 Casos de prueba

ID	Tipo	Precondiciones	Pasos	Resultado esperado	Resultado obtenido
CP-01	Unitaria	Módulo ms_auth importable	Validar NIT válidos (12345679, 1234567-9, CF, DPI de 13 dígitos) e inválidos	Válidos = verdadero; inválidos = falso	Aprobado: 8/8 combinaciones correctas
CP-02	Unitaria	Librería bcrypt instalada	Evaluar contraseñas fuertes y débiles; generar y verificar hash	Solo acepta 8+ caracteres con mayúscula, minúscula y número; el hash no contiene la clave	Aprobado: 4 claves débiles rechazadas, hash $2b$ verificado
CP-03	Unitaria	Tabla de transiciones definida	Probar Activo→Moroso, Moroso→Activo, Inactivo→Moroso, Activo→Activo	Solo se permiten las transiciones del modelo de negocio	Aprobado
CP-04	Unitaria	—	Normalizar MAC con guiones, sin separador y formatos inválidos	Formato AA:BB:CC:DD:EE:FF; inválidos lanzan error	Aprobado: 2 normalizadas, 4 rechazadas
CP-05	Unitaria	Umbrales GPON −27 / −8 dBm	Clasificar −19.5, −26, −28.2, −7.5, −27 y sin lectura	Normal, Advertencia, Crítica baja, Crítica alta, Advertencia, Sin lectura	Aprobado: 6/6
CP-06	Unitaria	—	Sugerir prioridad a partir del texto del reporte y consultar SLA	Sin internet = Crítica (4 h); lento = Alta (8 h); consulta = Baja (72 h)	Aprobado
CP-07	Unitaria	Factura de Q299.00	Validar montos 299, 150, 300, 0, −5 y texto	Solo el monto exacto es aceptado	Aprobado (falló en la ejecución de regresión simulada, ver 3.5)
CP-08	Unitaria	—	Generar 500 IDs de transacción y validar métodos de pago	500 IDs únicos con prefijo FC-; Bitcoin rechazado	Aprobado: 500/500 únicos
CP-09	Integración	BD creada con 01_schema.sql y 02_seed.sql	Cambiar estado del cliente 2 de Activo a Moroso	trg_auditar_cliente inserta CAMBIO_ESTADO_SERVICIO con estado anterior y nuevo en JSONB	Aprobado
CP-10	Integración	BD creada	Insertar factura de Q199 y su pago	trg_procesar_pago_exitoso cambia la factura a Pagado	Aprobado
CP-11	Integración	ONU con MAC 48:57:DD:A1:B2:C3 existente	Insertar otra ONU con la misma MAC	PostgreSQL rechaza con UniqueViolation	Aprobado
CP-12	Integración	BD creada	Crear ticket y ejecutar CALL sp_cerrar_ticket(id, 3)	Estado Cerrado, fecha_cierre registrada y excluido de vw_tickets_pendientes	Aprobado
CP-13	Integración	ONUs de prueba cargadas	Consultar vw_onus_senal_critica y ejecutar sp_actualizar_estado_onu	Aparece la ONU de −28.20 dBm y no la de −19.50; estado actualizado a Online	Aprobado
CP-14	Sistema	4 contenedores levantados	GET /health en puertos 3000–3003	HTTP 200, estado OK, db verdadero y esquema correcto por servicio	Aprobado: 4/4 servicios (66–71 ms)
CP-15	Sistema	Usuario técnico de prueba	POST /api/auth/login con clave correcta e incorrecta	200 con JWT y rol Tecnico; 401 con clave incorrecta	Aprobado
CP-16	Sistema	Factura pendiente de Q299 y token de técnico	POST /api/pagos por el total en efectivo	201; factura queda Pagado; ID de transacción FC-…	Aprobado (92 ms)
CP-17	Sistema	Factura pendiente	Cobrar Q100, luego Q299, luego Q299 otra vez	422 para el parcial, 201 para el total, 409 para el duplicado	Aprobado (falló en la ejecución de regresión simulada, ver 3.5)
CP-18	Sistema	Token de técnico	POST /api/onus con MAC en minúsculas y guiones; repetir	201 con MAC normalizada y señal Normal; 409 al repetir	Aprobado

ID	Tipo	Precondiciones	Pasos	Resultado esperado	Resultado obtenido
CP-19	Sistema	Token de técnico	Crear ticket «sin internet, fibra cortada» y cerrarlo	Prioridad Crítica, SLA 4 h; cierre con estado Cerrado	Aprobado
CP-20	Aceptación (Selenium)	Portal en ventana de 420 px, factura pendiente	Ingresar como técnico, buscar cliente 1, cobrar en efectivo	La fila cambia a Pagado y se muestra la transacción	Aprobado: capturas en Figura 4
CP-21	Aceptación (Selenium)	Usuario suscriptor	Ingresar al portal con credenciales de suscriptor	Mensaje «Solo personal técnico…»; sección de cobro oculta	Aprobado: captura en Figura 4
CP-22	Seguridad	Servicios levantados	Llamar 4 endpoints sin token, con token firmado con otra clave y con payload alterado	401 en todos los casos	Aprobado
CP-23	Seguridad	Token de suscriptor (cliente 1)	Intentar cobrar, ver perfil y facturas del cliente 2; ver su propio perfil	403 en recursos ajenos; 200 en el propio sin password_hash	Aprobado
CP-24	Seguridad	Servicios levantados	Enviar ' OR '1'='1, admin'-- y DROP TABLE en login y filtros	401 en login, lista vacía en filtros, sin trazas de error; la tabla sigue existiendo	Aprobado: 3/3 cargas
CP-25	Rendimiento	Catálogo con 3 planes	200 GET /api/planes secuenciales	0 errores y p95 menor a 300 ms	Aprobado: promedio 12.5 ms, p95 13.7 ms
CP-26	Rendimiento	Token de técnico	200 consultas de facturas con 20 hilos concurrentes	0 errores y p95 menor a 900 ms	Aprobado: p95 228 ms, 0 errores
CP-27	Manual (UX)	Teléfono o ventana de 420 px	Revisar legibilidad, tamaño de botones y desplazamiento en el portal	Todo el flujo es usable con una mano y sin desplazamiento horizontal	Aprobado con observación: el encabezado ocupa dos líneas; se agenda ajuste en Sprint 3
CP-28	Manual (UX)	Portal abierto	Provocar clave incorrecta, cliente sin facturas y método sin autorización	Mensajes en español, claros y sin términos técnicos	Aprobado

4.7 Ejecución del plan de pruebas

La ejecución oficial se realizó el 29 de septiembre de 2026 con el mismo comando que usa el pipeline. Resumen:

Indicador	Resultado
Casos automatizados ejecutados	26 casos (43 ejecuciones con parámetros)
Aprobados / fallidos	43 / 0
Casos manuales (UX)	2 aprobados, 1 observación menor
Duración de la suite	11.2 segundos
Cobertura de código total	84 % (reglas de negocio: 91 % a 100 %)
Latencia del catálogo (CP-25)	promedio 12.5 ms, p95 13.7 ms
Concurrencia (CP-26)	20 hilos, 200 solicitudes, p95 228 ms, 0 errores
Defectos encontrados	SET time_zone en el script SQL (corregido) y encabezado del portal en dos líneas (Sprint 3)

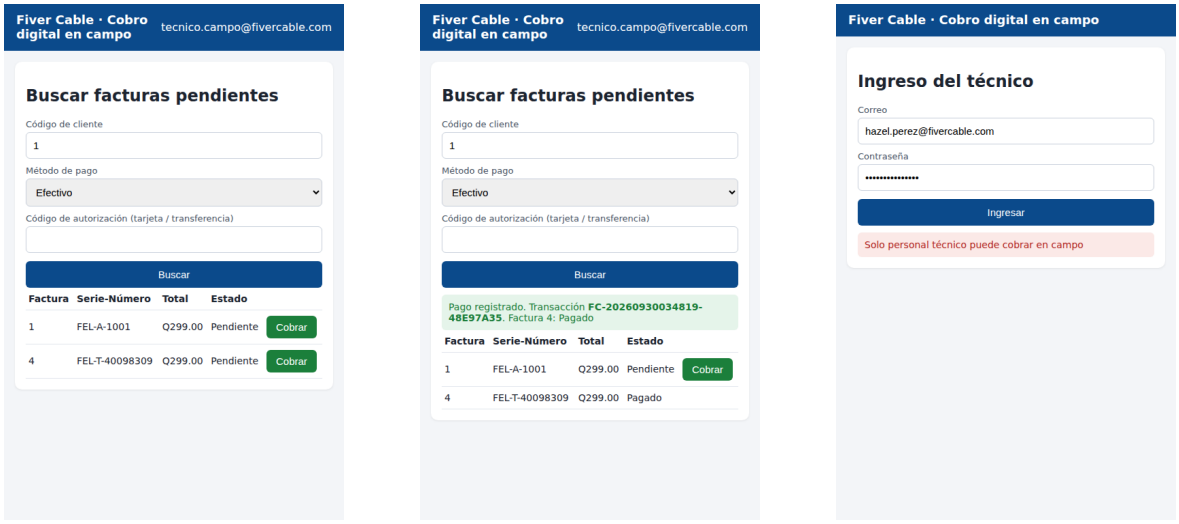
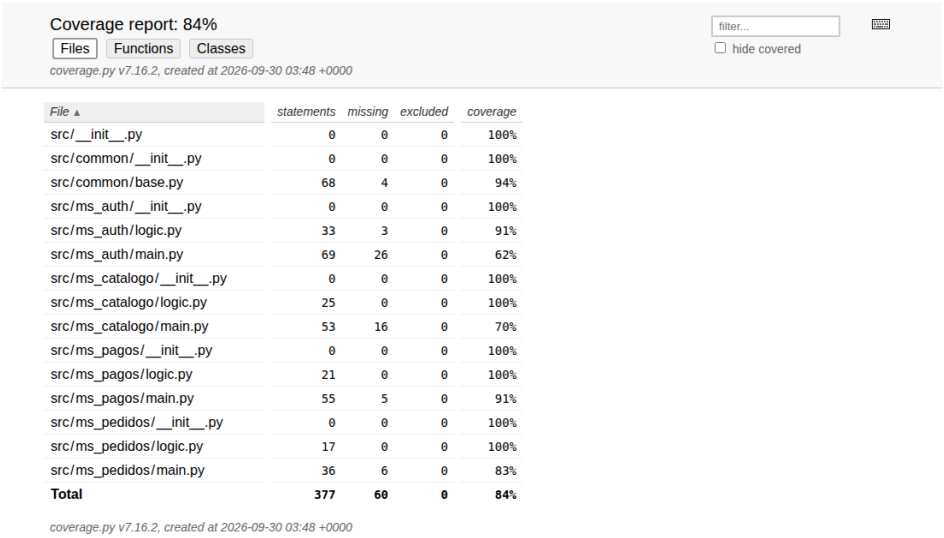


Figura 4. Capturas tomadas por Selenium: CP-20 antes y después del cobro (la factura pasa a Pagado) y CP-21 (suscriptor rechazado).



Coverage report: 84%

Files | Functions | Classes

filter... | hide covered

coverage.py v7.16.2, created at 2026-09-30 03:48 +0000

File ▲	statements	missing	excluded	coverage
src/__init__.py	0	0	0	100%
src/common/__init__.py	0	0	0	100%
src/common/base.py	68	4	0	94%
src/ms_auth/__init__.py	0	0	0	100%
src/ms_auth/logic.py	33	3	0	91%
src/ms_auth/main.py	69	26	0	62%
src/ms_catalogo/__init__.py	0	0	0	100%
src/ms_catalogo/logic.py	25	0	0	100%
src/ms_catalogo/main.py	53	16	0	70%
src/ms_pagos/__init__.py	0	0	0	100%
src/ms_pagos/logic.py	21	0	0	100%
src/ms_pagos/main.py	55	5	0	91%
src/ms_pedidos/__init__.py	0	0	0	100%
src/ms_pedidos/logic.py	17	0	0	100%
src/ms_pedidos/main.py	36	6	0	83%
Total	**377**	**60**	**0**	**84%**

coverage.py v7.16.2, created at 2026-09-30 03:48 +0000

Figura 5. Reporte de cobertura de código (84 % total).

4.8 Parámetros del sistema

Parámetro	Entorno de pruebas (CI)	Entorno de staging en Azure
Versión del sistema	Fiver Cable 2.0.0 (Sprint 2)	Fiver Cable 2.0.0, imagen con tag = número de build
Sistema operativo	Ubuntu 24.04 LTS, kernel 6.x, x86_64	Linux (Azure Container Apps), imagen python:3.11-slim
Lenguaje y librerías	Python 3.11.15, FastAPI 0.142.1, psycopg 3.3.6	Iguales (requirements.txt fijado)
Base de datos	PostgreSQL 16.13	Azure Database for PostgreSQL 16 Flexible, B1ms
Navegador	Google Chrome 148 headless, ventana de 420 × 900 px	Chrome / navegador del teléfono del técnico
Herramientas de prueba	pytest 9.1.1, pytest-html 4.2.0, pytest-cov 7.1.0, Selenium 4.49.0	Smoke tests con pytest
Hardware	2 vCPU Intel Xeon 2.10 GHz, 7.8 GB RAM	0.25 vCPU y 0.5 GB por contenedor, 0 a 2 réplicas
Red	Servicios en localhost (puertos 3000–3003)	HTTPS público por servicio, PostgreSQL con TLS y firewall

5. Evidencia Scrum del Sprint 2

Sprint 2: del sábado 19 al sábado 26 de septiembre de 2026, con correcciones finales hasta el martes 29. Objetivo del sprint: tener el proceso de cobro digital en campo funcionando con código real, pruebas automatizadas y despliegue continuo a staging.

5.1 Sprint Backlog (tareas de 16 horas o menos)

ID	Tarea	Responsable	Horas	Estado
T-01	Definir stack libre (Python 3.11, FastAPI, PostgreSQL 16) y estructura de /src	Scrum Master	6	Terminada
T-02	Versionar el script del Entregable 8 en db/init y corregir SET TIME ZONE	Dev 1	4	Terminada
T-03	Implementar ms_auth: registro, login JWT, roles y estado del suscriptor	Dev 1	12	Terminada
T-04	Implementar ms_catalogo: planes, inventario de ONUs y señal óptica	Dev 2	10	Terminada
T-05	Implementar ms_pedidos: tickets con prioridad automática y SLA	Dev 2	8	Terminada
T-06	Implementar ms_pagos y portal de cobro digital en campo	Dev 3	14	Terminada
T-07	Pruebas unitarias e integración (CP-01 a CP-13)	Dev 1	10	Terminada
T-08	Pruebas de sistema, seguridad y rendimiento (CP-14 a CP-26)	Dev 2	10	Terminada
T-09	Pruebas de aceptación con Selenium (CP-20, CP-21)	Dev 3	6	Terminada
T-10	Pipeline Azure (Build, Test, Deploy Staging) y Jenkinsfile	Dev 3	12	Terminada
T-11	Script de despliegue en Azure Container Apps	Scrum Master	8	Terminada
T-12	Documento DEVOPS 2 y Plan de Pruebas	Product Owner	12	Terminada
T-13	Grabar y publicar video YouTube #2	Todo el equipo	6	Terminada

Total estimado: 118 horas.

5.2 Burn down chart

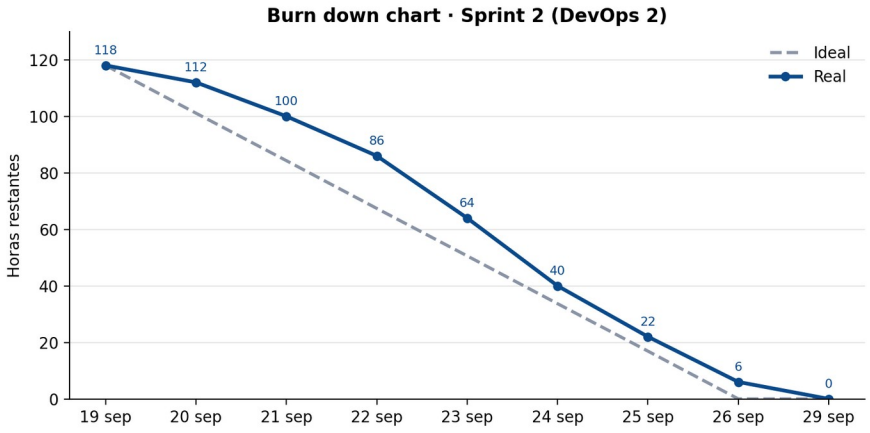


Figura 6. Burn down chart del Sprint 2 (horas restantes por día).

5.3 Daily Scrum

Fecha	¿Qué hice ayer?	¿Qué haré hoy?	¿Qué me bloquea?
Lun 21 sep	Definición del stack y versionado del script SQL	ms_auth y ms_catalogo	El script del Entregable 8 falla en el pipeline por SET time_zone
Mié 23 sep	ms_auth, ms_catalogo, ms_pedidos y corrección del script	ms_pagos, portal y pruebas unitarias	Ninguno
Vie 25 sep	Portal de cobro, pruebas de integración y sistema	Selenium, pipeline de Azure y script de staging	Definir la service connection de Azure para el despliegue
Sáb 26 sep	Pipeline completo y pruebas de seguridad y rendimiento	Documento DEVOPS 2 y video	Ninguno

5.4 Revisión del Sprint

Se demostró el cobro de una factura desde el portal en tamaño de teléfono, el pipeline con sus cuatro etapas y el reporte de pruebas. Se cumplió el objetivo del sprint: 13 de 13 tareas terminadas, 26 casos automatizados aprobados y 84 % de cobertura. Quedan para el Sprint 3 la integración con el bus de eventos para la reconexión automática y el ajuste visual del encabezado del portal.

5.5 Retrospectiva

¿Qué salió bien?	¿Qué podemos mejorar?	Acciones para el Sprint 3
Separar reglas de negocio de la API permitió pruebas unitarias rápidas	Las pruebas se escribieron después del código en algunos servicios	Escribir los casos de prueba junto con cada historia de usuario
El pipeline encontró un defecto real en el script SQL	La nomenclatura entre documentos se desalineó en DevOps 1	Revisar la matriz de trazabilidad en cada Pull Request
El reporte HTML facilita revisar resultados sin ejecutar nada	Poca holgura al final del sprint	Reservar un día de estabilización antes de la entrega

6. Video YouTube #2: guion

El video se publica en el canal del equipo junto al video de presentación y al de DevOps 1. Duración objetivo: 9 minutos.

Tiempo	Sección	Contenido y demostración
0:00 – 1:00	Introducción	Presentación del equipo, recordatorio de la arquitectura de Fiver Cable y objetivo de DevOps 2
1:00 – 2:30	Código y ramas	Estructura del repositorio, un microservicio (logic.py y main.py) y el flujo feature → develop → main
2:30 – 5:00	Demostración CI/CD	Hacer un commit, mostrar en Azure Pipelines las etapas Checkout, Build, Test y Deploy Staging, la pestaña Tests y el artefacto con el reporte HTML
5:00 – 6:00	Ejecución fallida	Rama con la regresión del monto parcial: el pipeline falla en CP-07 y CP-17 y no despliega
6:00 – 8:00	Plan de pruebas	Tipos de prueba, casos, ejecución de Selenium en vivo cobrando una factura, cobertura y parámetros del sistema
8:00 – 9:00	Conclusiones	Qué se logró, defecto encontrado por el pipeline y siguientes pasos del Sprint 3

7. Conclusiones

- El pipeline de Azure convierte cada commit en una verificación completa: si una prueba falla, el cambio no avanza a staging, como se comprobó con la regresión simulada del monto de cobro.
- Automatizar la creación de la base de datos reveló un defecto que la ejecución manual en pgAdmin no mostró, lo que confirma que la calidad se hace visible cuando se mide de forma repetible.
- El plan de pruebas cubre seis tipos de prueba automatizados más pruebas manuales de usabilidad, con resultados trazables por ID de caso y una línea base de 84 % de cobertura y p95 de 13.7 ms.
- El proceso de cobro digital en campo ya funciona de extremo a extremo y sirve como base para las siguientes etapas: reconexión automática por eventos, integración FEL real y la aplicación móvil.

Anexo A. Configuración en Azure DevOps

- Pipelines → New pipeline → GitHub → fiver-cable-devops → Existing Azure Pipelines YAML file → /azure-pipelines.yml.
- Pipelines → Environments → New environment: fivercable-staging (opcional: agregar aprobación manual).
- Project settings → Service connections → Azure Resource Manager, nombre fivercable-azure, sobre la suscripción Azure for Students.

- Pipeline → Edit → Variables: PG_ADMIN_PASSWORD y JWT_SECRET_STAGING marcadas como secretas.
- Repos → Branches → develop y main → Branch policies: exigir el pipeline como validación de build en Pull Requests.
- Evidencia de ejecución fallida: crear la rama feature/demo-fallo, cambiar en src/ms_pagos/logic.py la condición «pagado != total» por «pagado > total», hacer push y abrir un Pull Request; el pipeline queda en rojo en la etapa Build (CP-07).